

REST Create and Retrieve Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-flask-restful-cr-lab>  <https://github.com/learn-co-curriculum/python-p4-flask-restful-cr-lab/issues/new>

Learning Goals

- Build RESTful APIs that are easy to navigate and use in applications.
- Develop a Flask API with successful frontend connections via `fetch()`.
- Integrate create and retrieve routes with the associated actions to return the appropriate JSON data.

Key Vocab

- **Representational State Transfer (REST)**: a convention for developing applications that use HTTP in a consistent, human-readable, machine-readable way.
- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- **PUT** : an HTTP request method that signifies that the client is attempting to update a resource with new information contained in a complete record.
- **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.

Introduction

In this lab, we'll be building an API for a plant store! In addition to our usual Flask code, there is code for a React frontend application in the `client` directory.

The code for the frontend application is done. Your job is to create the Flask API so that the `fetch` requests on the frontend work successfully.

Instructions

The React application is in the `client` directory. To set it up, from the root directory, run:

```
$ npm install --prefix client
```

Using `--prefix client` will run the npm command within the `client` directory.

To set up your backend, run:

```
$ pipenv install; pipenv shell
```

Then navigate to the `server/` directory to run your Python code.

First, you will need to set up your database. Go ahead and run the following command to create the `instance/app.db` database file:

```
$ flask db upgrade head
```

To see how the React application and Flask API are interacting, first, you will need to set the default port number to match the proxy setup in the client's package.json. In this case, the port number is 5555.

```
export FLASK_RUN_PORT=5555
```

Now you can run the Flask application in one terminal by running:

```
$ flask run
```

Then, [open another terminal](https://code.visualstudio.com/docs/editor/integrated-terminal#_managing-terminals) and run React:

```
$ npm start --prefix client
```

Each application will run on its own port on `localhost` :

- React: <http://localhost:4000>
- Flask: <http://localhost:5555>

Take a look through the components in the `client/src/components/` folder to get a feel for what our app does. Note that the `fetch` requests in the frontend (in `NewPlantForm` and `PlantPage`) don't include the backend domain:

```
fetch("/plants");  
// instead of fetch("http://localhost:5000/plants")
```

This is because we are [proxying](https://create-react-app.dev/docs/proxying-api-requests-in-development/) these requests to our Flask API.

Deliverables

Model

Edit the `Plant` model in `models.py` to match this specification:

Column Name	Data Type
name	string
image	string

Column Name	Data Type
price	decimal

After defining the columns for the **Plant** model and saving the file, do the following to update and seed the plant table:

1. Create a revision that tracks your changes to models.py

```
$ flask db revision --autogenerate -m'add columns to table'
```

1. Upgrade the db to the latest revision

```
$ flask db upgrade head
```

1. Seed the database

```
$ python seed.py
```

Routes

Your API should have the following routes as well as the associated controller actions that return the appropriate JSON data:

Index Route

GET /plants

Response Body

```
[  
  {  
    "id": 1,  
    "name": "Aloe",  
    "image": "../images/aloe.jpg",
```

```
    "price": 11.50
  },
  {
    "id": 2,
    "name": "ZZ Plant",
    "image": "./images/zz-plant.jpg",
    "price": 25.98
  }
]
```

Show Route

GET /plants/:id

Response Body

```
-----
{
  "id": 1,
  "name": "Aloe",
  "image": "./images/aloe.jpg",
  "price": 11.50
}
```

Create Route

POST /plants

Headers

```
-----
Content-Type: application/json
```

Request Body

```
-----  
{  
  "name": "Aloe",  
  "image": "./images/aloe.jpg",  
  "price": 11.50  
}
```

Response Body

```
-----  
{  
  "id": 1,  
  "name": "Aloe",  
  "image": "./images/aloe.jpg",  
  "price": 11.50  
}
```

Note 1: When adding image URLs, you will need to use absolute URLs from the internet; we have only uploaded the two images to this project directory.

Note 2: Due to the structure of the client, you will need to use the `get_json()` method to retrieve data for the create route. When you write your own clients, you can decide whether data is passed to the backend via forms or raw JSON.

Once all the tests are passing, start up the React app and explore the functionality to see how the routes you created are being used.

Resources

- [What RESTful Actually Means](https://codewords.recurse.com/issues/five/what-restful-actually-means) ↗ <https://codewords.recurse.com/issues/five/what-restful-actually-means>
- [Flask-RESTful](https://flask-restful.readthedocs.io/en/latest/) ↗ <https://flask-restful.readthedocs.io/en/latest/>
- [HTTP request methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods) ↗ <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>

- [Proxying API Requests in Development - React](https://create-react-app.dev/docs/proxying-api-requests-in-development/)  [\(https://create-react-app.dev/docs/proxying-api-requests-in-development/\)](https://create-react-app.dev/docs/proxying-api-requests-in-development/)

This tool needs to be loaded in a new browser window

Load REST Create and Retrieve Lab (CodeGrade) in a new window